

CSC123 — Inheritance Project

ASCII Shape Parade

Overview

You are given a working ASCII animation program. Shapes bounce around a terminal window in color. Shapes of the SAME color pass through each other. Shapes of DIFFERENT colors bounce off each other.

Your job is to restore missing pieces of code that were removed from several classes. Each task focuses on one core inheritance concept.

Run the demo first so you know what the finished program looks like, then work through the tasks.

Running the Demo

The instructor has provided compiled .class files so you can run the finished program before you start coding.

Steps

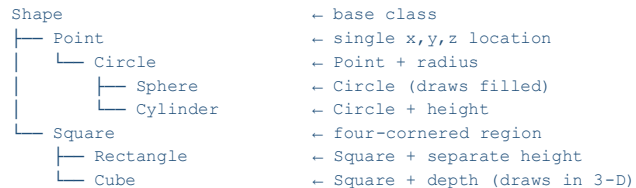
- Put all the provided files (.class) in the same folder
- Open a terminal in that folder
- Run:
`java Driver`
- Watch for 30 seconds. Notice:
 - RED shapes (Circle, Sphere, Cylinder) pass through each other
 - GREEN shapes (Square, Cube, Rectangle) pass through each other
 - A RED shape and a GREEN shape BOUNCE off each other

After the animation, the program prints an equals() demo showing which comparisons return true vs false.

Note: you will need Java in your PATH environment variable. If it is not set up, watch this video:
<https://www.youtube.com/watch?v=0y5SwMSBKNY>

Class Hierarchy

Read this before starting. Every class in this project fits into one of two branches:



Notice the two branches: the Point branch (shapes with a center location) and the Square branch (shapes with four corners). This is why Circle extends Point but Square does not.

Files You Are Given

File	Edit?	Purpose
Shape.java	Task 2	Base class — all shapes inherit from this
Point.java	Task 1	A 3-D point; draws as *
Circle.java	Task 1	A circle (IS-A Point with radius)
Sphere.java	Tasks 1&3	A sphere (IS-A Circle); draws filled
Cylinder.java	Task 1	A cylinder (IS-A Circle with height)
Square.java	Task 1	A square; draws as # outline
Rectangle.java	Tasks 1&4	A rectangle (IS-A Square with separate height)
Cube.java	Task 1	A cube (IS-A Square with depth); draws in 3-D
Driver.java	No	Runs the animation — do not change this
AsciiCanvas.java	No	The character grid you draw onto
Color.java	No	Color constants (RED, GREEN, etc.)
Ansi.java	No	Terminal escape sequences
MovePattern.java	No	Movement policy (bounce, wrap, random walk)

You are to submit only the files that you have changed (do not zip them up)

Task 1 — Restore getName() in every shape class

Task 1: Restore getName() in all shape classes [Easy]

File: Point.java, Circle.java, Sphere.java, Cylinder.java, Square.java, Rectangle.java, Cube.java **Concept:** *Overriding a method in a subclass*

The base class Shape has a getName() method that returns "Shape". Every subclass should override it to return its own name. The method bodies have been removed. Without them the legend bar at the bottom of the animation shows "Shape" for every shape instead of their real names.

What to write:

1. In each of the seven files, find the stub: `public String getName() { ... }`
2. Return the correct string — e.g., in Circle.java return `"Circle"`

How to test

- Compile and run — the legend bar at the bottom should show "Circle Sphere Cylinder Square Cube Rectangle" in their colors

Why this matters

Every class in the hierarchy overrides the same method signature but returns a different value. This is the simplest form of polymorphism: one method call (getName()) produces different output depending on which object receives it.

Task 2 — Restore equals() in Shape.java

Task 2: Restore equals() in Shape [Medium]

File: `Shape.java`

Concept: *Overriding a method from Object*

The `equals()` method decides the collision rule. Right now it is missing its body, so every shape bounces off every other shape — the pass-through effect is broken. Write the body so that two shapes are equal when they have the same `Color`.

What to write:

1. Open `Shape.java` and find the stub: `public boolean equals(Object obj) { ... }`
2. Check `obj == this`
3. Check `obj` for `null`
4. Check `obj instanceof Shape`
5. Cast `obj` to `Shape`
6. Return true if and only if both shapes have the same color

How to test

- Compile and run — RED shapes should now pass through each other, GREEN through GREEN, RED bounces off GREEN
- The `equals()` printout at the end of the animation should show true for same-color pairs
- Test for all possible inputs (this is some you should ALWAYS do)

Hint

The color field is type `Color` (an enum). You can compare enum values with `==`:

```
return this.color != null && this.color == other.color;
```

Task 3 — Restore draw() in Sphere.java

Task 3: Restore draw() in Sphere [Medium]

File: `Sphere.java`

Concept: *Overriding draw() — inherited vs overridden behavior*

Sphere extends Circle. Circle draws a hollow ring of 'o' characters. Sphere should draw a FILLED ball using 'O' on the outer edge and '.' in the interior. The draw() method has been removed from Sphere. Without it, Sphere inherits Circle's draw() and looks like a ring instead of a solid ball.

What to write:

1. In `Sphere.java` write a `draw(AsciiCanvas c)` method that overrides Circle's version
2. Get the integer radius: `int r = rInt();` (inherited from Circle)
3. Calculate screen center: `int cx = sx + r; int cy = sy + r;`
4. Loop `dy` and `dx` from `-r` to `+r`
5. For each cell inside the circle ($dx^2 + dy^2 \leq r^2$), plot 'O' on the outer edge, and '.' in the interior

How to test

- Before you write draw(): run the program and observe that Sphere looks like a hollow ring (same as Circle)
- After you write draw(): Sphere should look like a filled ball

Key formula

```
int d2 = dx*dx + dy*dy;
if (d2 <= r*r) {
    char ch = (Math.abs(d2 - r*r) <= r+1) ? 'O' : '.';
    c.put(cx+dx, cy+dy, ch, col);
}
```

Again, use `rInt()` (inherited from Circle) for the integer radius, and `sx+r` / `sy+r` for the screen center.

Task 4 — Restore footprintW/H() and draw() in Rectangle.java

Task 4: Restore footprintW/H() and draw() in Rectangle [Hard]

File: `Rectangle.java`

Concept: *Overriding multiple methods, using fields from parent classes*

Rectangle extends Square. Square has one dimension: side. Rectangle adds a second: height2. Three methods have been removed: footprintW(), footprintH(), and draw(). Without them, Rectangle inherits Square's versions, which treat it as a square and draw it with # instead of the correct +===+ style.

What to write:

1. Write `footprintW()` — return `sInt()` (the width, inherited from Square)
2. Write `footprintH()` — return `hInt2()` (a private helper already in the file)
3. Write `draw(AsciiCanvas c)` — top/bottom edges with '=', side edges with '|', corners with '+'

How to test

- Before: Rectangle looks like a square drawn with # characters
- After: Rectangle is wider than tall, drawn with +===+ style borders

Fields available from parent classes

<code>sx, sy</code>	← screen position	(from Shape)
<code>side</code>	← width	(from Square — use <code>sInt()</code> for int version)
<code>height2</code>	← height	(in Rectangle — use <code>hInt2()</code> for int version)

Compiling and Running

Compile all Java files at once:

```
javac *.java
```

Then run:

```
java Driver
```

The animation runs for ~600 frames (about 36 seconds), then prints the equals() demo and exits.

Grading

Task	Description	Points	Notes
1	getName() — all 7 classes return the correct name	10	<i>run program, check legend bar</i>
2	equals() — pass-through collision works correctly	10	<i>same-color shapes pass through</i>
3	Sphere.draw() — sphere looks filled, not hollow	5	<i>visual check</i>
4	Rectangle footprint + draw() — correct shape and style	5	<i>visual check + collision bounds</i>

Required File Header

Every .java file you submit must begin with:

```
/* *****  
 * Name:  
 * Course: CSC123  
 * Lab:  
 * Date:  
 * File:  
 * Description:  
 ***** */
```

Missing header → formatting deduction.

Academic Integrity & AI Usage Policy

Please read carefully — violations may result in a failing grade or disciplinary action.

1. The One Rule That Covers Everything

Every character of code and every comment you submit must be typed by your own hands, from your own understanding, in your own way. No exceptions.

You are free — and encouraged — to consult any resource you like: textbooks, documentation, websites, AI tools, classmates, tutors. The goal of all that research is to build your understanding. Once you understand what to do, you close those resources and write the solution yourself, in your own words.

Copying is plagiarism regardless of whether you disclose it, cite the source, or understand the code you copied. Disclosure does not make copying acceptable.

2. What Counts as Plagiarism

The following are violations, with or without citation or disclosure:

- Copying code from any source — another student, a website, a book, a forum, or an AI — and submitting it as your own.
- Adapting or lightly modifying someone else's code structure and submitting it as your own.
- Allowing an AI tool (ChatGPT, Copilot, Claude, Gemini, etc.) to write or complete any portion of your code.
- Sharing your own working solution with another student so they can use it as a reference while writing theirs.
- Submitting work written collaboratively with another student unless the assignment explicitly requires it.

3. AI Tools — The Right Way to Use Them

AI assistants are powerful learning tools and you are encouraged to use them — but only to build understanding, never to produce code you submit.

✔ Encouraged	✗ Always a Violation
Ask AI to explain what a method or concept does	Ask AI to write a function for your assignment
Ask AI to explain why a certain approach works	Ask AI to design your solution
Ask AI to help you understand a compiler error	Paste in AI-generated code, even with edits
Ask AI to explain inheritance or polymorphism	Use GitHub Copilot or any autocomplete to finish a line
Use AI to quiz yourself on concepts before coding	Accept any AI-suggested completion — even a comment

4. A Special Note on IDE Autocomplete

Accepting an AI-powered autocomplete suggestion — even to finish a single line or a comment — is a violation of this policy. Disable GitHub Copilot, Tabnine, and similar AI completion tools while working on course assignments.

This includes completions in VS Code, JetBrains IDEs, and any other editor with an AI plugin. Traditional, non-AI autocomplete (such as standard IntelliSense that only suggests symbols defined in your own code) is fine.

5. Consequences of Violations

Violations of this policy may result in:

- A zero on the assignment with no opportunity to resubmit.
- Repeated violations may result it negative a score
- Repeated violations may result in referral to the Office of Academic Affairs for formal disciplinary review.
- Repeated violations may result a failing grade for the course.
- Repeated violations may result a permanent notation on your academic record and a meeting with Computer Science Department Chair.

